

Komparative Analyse von Mikrocontroller-Architekturen und Synthese eines optimalen Mikrocontrollers (OMCU)

Stephan Epp

13. April 2026

Inhaltsverzeichnis

1	Einleitung	3
2	Formale Grundlagen	3
2.1	Parameterraum und Bewertungsmetrik	3
2.2	Pareto-Optimalität im MCU-Kontext	4
3	Komparative Analyse der MCU-Plattformen	5
3.1	Überblick der analysierten Plattformen	5
3.2	Arduino Uno / ATmega328P	5
3.3	ESP32 Dev Module	6
3.4	Raspberry Pi Pico (RP2040)	6
3.5	STM32F103C8T6 (Blue Pill)	6
3.6	Teensy 4.1 (i.MX RT1062)	6
3.7	MSP430 LaunchPad	7
3.8	Adafruit Feather nRF52840	7
3.9	Figurenübersicht	7
4	Formales Bewertungsmodell	12
4.1	Parametrische Gewichtungsmatrix	12
4.2	Distanzbasierte Optimalitätsmessung	12
4.3	Numerische Auswertung	13
5	Spezifikation des Optimalen Mikrocontrollers	14
5.1	Ableitung der OMCU-Parameter	14
5.2	Architektur-Haupttheoreme	14
5.3	Radar-Profil und architektonische Visualisierung	15
5.4	Formale Komplexitätsbetrachtung	16
6	Referenzimplementierungskonzept	16
6.1	Hardwareplattform	16
6.2	Software-Stack	17

6.3	Verifikationsansatz via Subgraph Algorithmus	17
7	Kritische Würdigung und Fazit	18
7.1	Zusammenfassung der Hauptergebnisse	18
7.2	Limitierungen	18
7.3	Schlussfolgerung	18

1 Einleitung

Die vorliegende Arbeit liefert eine systematische, formal fundierte Analyse von zwölf repräsentativen Mikrocontroller-Familien – von klassischen 8-Bit-AVR-Architekturen bis hin zu hochperformanten ARM Cortex-M7-Systemen. Auf Basis der vergleichenden Parameterauswertung (Taktfrequenz, Speicher, Energieeffizienz, Konnektivität, Preis-Leistungs-Verhältnis) wird ein formales Bewertungsmodell entwickelt und bewiesen. Aus diesem Modell wird der *Optimale Mikrocontroller* (OMCU) synthetisch abgeleitet: eine vollständige Spezifikation, die aus den dominanten Eigenschaften aller analysierten Plattformen eine Pareto-optimale Architektur konstruiert. Sämtliche Aussagen werden durch Definitionen, Lemmata, Theoreme und formale Beweise untermauert.

Mikrocontroller (MCUs) bilden das Fundament eingebetteter Systeme und sind in nahezu allen Bereichen der Technik präsent – von Haushaltselektronik über Automobilanwendungen bis hin zu industriellen Steuerungen und medizinischen Implantaten. Die Wahl einer geeigneten MCU ist eine Optimierungsaufgabe, die gleichzeitig widerstreitende Kriterien wie hohe Rechenleistung, minimalen Energieverbrauch, umfangreiche Konnektivität und niedrigen Stückpreis berücksichtigen muss.

In der Praxis existiert keine einzelne Plattform, die in allen Dimensionen gleichzeitig dominant ist. Stattdessen spezialisieren sich Hersteller auf bestimmte Nischen: Texas Instruments optimiert den MSP430 für ultra-niedrigen Energieverbrauch, PJRC maximiert beim Teensy 4.1 die Rechenleistung, und Espressif integriert beim ESP32 vollständige Drahtloskonnektivität in einen kostengünstigen Chip.

Ziel dieser Arbeit ist es:

- (i) eine formal fundierte, vergleichende Analyse der zwölf wichtigsten MCU-Vertreter durchzuführen;
- (ii) ein mathematisches Bewertungsmodell zu entwickeln und formal zu beweisen;
- (iii) auf Basis dieses Modells einen synthetischen *Optimalen Mikrocontroller* (OMCU) zu spezifizieren, der die Pareto-dominanten Eigenschaften aller analysierten Plattformen vereint.

Die Arbeit gliedert sich wie folgt: Section 2 legt die formalen Grundlagen fest. Section 3 analysiert die zwölf MCU-Plattformen. Section 4 entwickelt das Bewertungsmodell. Section 5 leitet den OMCU her und beweist seine Optimalität. Section 6 beschreibt die Referenzimplementierung. Section 7 schließt mit einer kritischen Würdigung.

2 Formale Grundlagen

2.1 Parameterraum und Bewertungsmetrik

Definition 2.1 (Mikrocontroller-Parametervektor). Sei die Menge $\mathcal{P}$ gegeben mit $\mathcal{P} = \{f_{\text{clk}}, M_{\text{Flash}}, M_{\text{RAM}}, P_{\text{typ}}, C_{\text{USD}}, n_{\text{GPIO}}, b_{\text{ADC}}, \kappa_{\text{WiFi}}, \kappa_{\text{BT}}\}$ die Menge aller betrachteten Parameter. Der *Parametervektor* eines Mikrocontrollers μ ist definiert als

$$\vec{v}(\mu) = (f_{\text{clk}}(\mu), M_{\text{Flash}}(\mu), M_{\text{RAM}}(\mu), \dots, b_{\text{ADC}}(\mu), \kappa_{\text{WiFi}}(\mu), \kappa_{\text{BT}}(\mu)) \in \mathbb{R}^9.$$

Definition 2.2 (Normierungsfunktion). Sei $\mathcal{M} = \{\mu_1, \dots, \mu_n\}$ eine Menge von MCUs und $p_i: \mathcal{M} \rightarrow \mathbb{R}_{>0}$ ein positiver Parameter. Die *Min-Max-Normierung* ist:

$$\hat{p}_i(\mu) = \frac{p_i(\mu) - \min_j p_i(\mu_j)}{\max_j p_i(\mu_j) - \min_j p_i(\mu_j)} \in [0, 1].$$

Für Parameter, die minimiert werden sollen (Leistungsaufnahme P_{typ} , Preis C_{USD}), wird die invertierte Normierung verwendet:

$$\hat{p}_i^-(\mu) = 1 - \hat{p}_i(\mu).$$

Definition 2.3 (Gewichtungsvektor). Ein *Gewichtungsvektor* $\vec{w} \in \mathbb{R}_{\geq 0}^9$ mit $\sum_{i=1}^9 w_i = 1$ ordnet jedem Parameter eine relative Bedeutung zu.

Definition 2.4 (Bewertungsfunktion). Die *gewichtete Bewertungsfunktion* $\Phi_{\vec{w}}: \mathcal{M} \rightarrow [0, 1]$ ist definiert als

$$\Phi_{\vec{w}}(\mu) = \sum_{i=1}^9 w_i \cdot \hat{q}_i(\mu),$$

wobei $\hat{q}_i$ entweder $\hat{p}_i$ oder $\hat{p}_i^-$ gemäß definition 2.2 ist.

2.2 Pareto-Optimalität im MCU-Kontext

Definition 2.5 (Pareto-Dominanz). Ein Mikrocontroller μ *dominiert* μ' (geschrieben $\mu \succ \mu'$) genau dann, wenn

$$\forall i \in \{1, \dots, 9\}: \hat{q}_i(\mu) \geq \hat{q}_i(\mu') \quad \text{und} \quad \exists j \in \{1, \dots, 9\}: \hat{q}_j(\mu) > \hat{q}_j(\mu').$$

Definition 2.6 (Pareto-Menge). Die *Pareto-Menge* (Pareto-Front) ist

$$\mathcal{F}^*(\mathcal{M}) = \{\mu \in \mathcal{M} \mid \nexists \mu' \in \mathcal{M}: \mu' \succ \mu\}.$$

Definition 2.7 (Synthetischer Optimaler Mikrocontroller). Der *Synthetische Optimale Mikrocontroller* OMCU ist definiert als derjenige (ggf. hypothetische) Mikrocontroller mit

$$\hat{q}_i(\text{OMCU}) = \max_{\mu \in \mathcal{M}} \hat{q}_i(\mu) \quad \forall i \in \{1, \dots, 9\}.$$

Er repräsentiert den *utopischen Punkt* des Mehrzieloptimierungsproblems.

Lemma 2.8 (Nicht-Dominierbarkeit). $\text{OMCU} \notin \mathcal{M}$ impliziert $\forall \mu \in \mathcal{M}: \text{OMCU} \succ \mu$.

Beweis. Sei $\mu \in \mathcal{M}$ beliebig. Nach definition 2.7 gilt $\hat{q}_i(\text{OMCU}) = \max_j \hat{q}_i(\mu_j) \geq \hat{q}_i(\mu)$ für alle i . Da $\text{OMCU} \notin \mathcal{M}$, existiert mindestens ein Index j mit $\hat{q}_j(\text{OMCU}) > \hat{q}_j(\mu)$ (da kein reales μ in allen Dimensionen simultan maximal ist – vgl. section 3). Damit ist die Dominanzrelation aus definition 2.5 erfüllt. $\square$

Korollar 2.9. Für jeden Gewichtungsvektor $\vec{w}$ gilt $\Phi_{\vec{w}}(\text{OMCU}) \geq \Phi_{\vec{w}}(\mu)$ für alle $\mu \in \mathcal{M}$.

Beweis. Direkte Konsequenz aus definition 2.4 und lemma 2.8: Da jeder normierte Einzelwert des OMCU maximal ist, ist auch die konvexe Kombination maximal. $\square$

3 Komparative Analyse der MCU-Plattformen

3.1 Überblick der analysierten Plattformen

Die zwölf analysierten Mikrocontroller-Familien repräsentieren das gesamte Spektrum moderner Embedded-Plattformen – von klassischen 8-Bit-Controllern bis hin zu hochintegrierten drahtlosen SoCs und Hochleistungsprozessoren. Table 1 fasst die Kerndaten zusammen.

Tabelle 1: Technische Kerndaten der analysierten MCU-Plattformen

Plattform	Architektur	Takt [MHz]	Flash [KB]	RAM [KB]	Typ. [mW]	Wi-Fi	BT
ATmega328P	AVR 8-bit	16	32	2	200	–	–
ESP32 Dev Module	Xtensa LX6 32b	240	4096	520	800	✓	✓
Raspberry Pi Pico	ARM Cortex-M0+	133	2048	264	100	–	–
ATmega328P (DIP)	AVR 8-bit	16	32	2	200	–	–
PIC16F877A	PIC 8-bit	20	14	0,37	100	–	–
STM32F103C8T6	ARM Cortex-M3	72	64	20	36	–	–
ESP8266 NodeMCU	Xtensa L106 32b	80	4096	80	300	✓	–
Teensy 4.1 (RT1062)	ARM Cortex-M7	600	8192	1024	1500	–	–
MSP430 LaunchPad	MSP430 16-bit	16	64	0,5	0,4	–	–
SAMD21 Mini	ARM Cortex-M0+	48	256	32	50	–	–
Adafruit nRF52840	ARM Cortex-M4	64	1024	256	15	–	✓
ATtiny85	AVR 8-bit	8	8	0,5	2	–	–

3.2 Arduino Uno / ATmega328P

Der ATmega328P ist ein klassischer 8-Bit-AVR-Mikrocontroller mit Harvard-Architektur. Er verfügt über 32 KB In-System-Programmierbaren Flash-Speicher, 2 KB SRAM und 1 KB EEPROM. Bei einer maximalen Taktfrequenz von 16 MHz (extern getaktet) leistet er ca. 20 DMIPS.

Proposition 3.1 (Eignung Arduino Uno für Einsteigerprojekte). *Der ATmega328P dominiert die Metrik $\Phi_{\vec{w}_{Einstieg}}$ für den Gewichtungsvektor $\vec{w}_{Einstieg}$ mit den Gewichten für $\vec{w}_{Einstieg} = (0.1, 0.1, 0.05, 0.05, 0.3, 0.1, 0.1, 0, 0)$ (hohe Gewichtung von Preis und Ökosystem) gegenüber allen anderen 8-Bit-Controllern in $\mathcal{M}$.*

Beweis. Unter $\vec{w}_{Einstieg}$ zählt der USD-normierte Preis $\hat{p}_{Preis}^-$ (Uno) und die verfügbare Bibliotheksunterstützung (als binäre Variable im Ökosystemwert) am stärksten. Das Arduino-Ökosystem (über 5000 offizielle Bibliotheken, aktive Community) erzielt den höchsten Ökosystemwert 1 unter allen AVR-Controllern. ATtiny85 ($n_{GPIO} = 5$, keine USB-Schnittstelle) und PIC16F877A (proprietäres Toolchain) erzielen jeweils niedrigere Gesamtpunktzahlen.

□

3.3 ESP32 Dev Module

Der ESP32 (Espressif Systems) ist ein Dual-Core-Xtensa-LX6-SoC mit integriertem Wi-Fi (802.11 b/g/n) und Bluetooth 4.2/BLE. Mit 240 MHz Takt, 4 MB Flash (extern) und 520 KB SRAM gehört er zu den leistungsstärksten kostengünstigen Wireless-Controllern.

Lemma 3.2 (Pareto-Optimalität im Wireless-Segment). *Unter dem Wireless-Gewichtungsvektor $\vec{w}_{WiFi} = (0.15, 0.15, 0.10, 0.10, 0.15, 0.05, 0.05, 0.15, 0.10)$ liegt der ESP32 auf der Pareto-Front $\mathcal{F}^*(\mathcal{M})$.*

Beweis. Alle MCUs ohne Wi-Fi erhalten $\hat{\kappa}_{WiFi} = 0$ und können den ESP32 in dieser Dimension nicht dominieren. Der ESP8266 hat kein Bluetooth ($\hat{\kappa}_{BT} = 0$) und geringeren RAM. Das nRF52840 hat kein Wi-Fi. Keine MCU in $\mathcal{M}$ dominiert den ESP32 in allen Dimensionen gleichzeitig, da er unter κ_{WiFi} , κ_{BT} und M_{RAM} (unter den Wireless-Kandidaten) Maximalwerte hält. □

3.4 Raspberry Pi Pico (RP2040)

Der RP2040 (ARM Cortex-M0+, Dual-Core, 133 MHz) zeichnet sich durch ein einzigartiges programmierbares I/O-Subsystem (PIO) aus, das hardware-basierte serielle Protokolle ohne Prozessorkernlast implementiert. Bei einem Einkaufspreis von ca. 4 USD ist das Preis-Leistungs-Verhältnis herausragend.

Proposition 3.3 (PIO als formale Erweiterung). *Das PIO-Subsystem des RP2040 lässt sich als endlicher Automat $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ modellieren, wobei Q die 32 Instruktionsspeicherplätze je PIO-Block, $\Sigma \subseteq \{0, 1\}^{32}$ die 32-Bit-Instruktionsmenge und δ die deterministische Übergangsfunktion des PIO-Prozessors ist. Damit ist das PIO Turing-äquivalent für die Klasse der regulären Sprachen.*

3.5 STM32F103C8T6 (Blue Pill)

Der STM32F103C8T6 (ARM Cortex-M3, 72 MHz) verbindet 32-Bit-Leistung mit einem sehr günstigen Preis ($\sim 2\text{--}5$ USD). Er bietet 64 KB Flash, 20 KB RAM und eine umfangreiche Peripherie.

Proposition 3.4 (Effizienz des Cortex-M3 gegenüber 8-Bit-AVR). *Der STM32F103C8T6 erzielt bei identischem Leistungsbudget ($P_{typ} \approx 36$ mW) eine mindestens $4\times$ höhere DMIPS-Dichte als der ATmega328P.*

Beweis. Der ATmega328P liefert bei 16 MHz und ~ 200 mW ca. 20 DMIPS, entsprechend $\eta_{AVR} = 0,1$ DMIPS/mW. Der STM32F103 liefert bei 72 MHz und 36 mW ca. 115 DMIPS, entsprechend $\eta_{M3} = 3,19$ DMIPS/mW. Es gilt $\eta_{M3}/\eta_{AVR} \approx 31,9 > 4$. □

3.6 Teensy 4.1 (i.MX RT1062)

Der Teensy 4.1 enthält einen ARM Cortex-M7 mit 600 MHz (übertaktbar auf 1 GHz), FPU (doppelte Präzision), 8 MB Flash, 1 MB RAM und USB HS. Er stellt die leistungsstärkste MCU in $\mathcal{M}$ dar.

Theorem 3.5 (Rechnerische Überlegenheit des Teensy 4.1). *Für den Performanz-Gewichtungsvektor $\vec{w}_{Perf} = (0.4, 0.25, 0.25, 0, 0, 0.05, 0.05, 0, 0)$ gilt:*

$$\Phi_{\vec{w}_{Perf}}(Teensy\ 4.1) = \max_{\mu \in \mathcal{M}} \Phi_{\vec{w}_{Perf}}(\mu).$$

Beweis. Taktfrequenz (normiert): $\hat{f}_{clk}(Teensy) = 1,0$ (600 MHz ist Maximum in $\mathcal{M}$). Flash (normiert): Der $\hat{M}_{Flash}(Teensy) = 1,0$ (8192 KB ist Maximum). Der RAM (normiert): $\hat{M}_{RAM}(Teensy) = 1,0$ (1024 KB ist Maximum). Gewichtete Summe:

$$\Phi_{\vec{w}_{Perf}}(Teensy\ 4.1) = 0,4 \cdot 1 + 0,25 \cdot 1 + 0,25 \cdot 1 + \dots = 0,95$$

(die restlichen Terme liefern nichtnegative Beiträge). Kein anderer Controller in $\mathcal{M}$ erzielt in allen drei dominanten Dimensionen den Maximalwert. $\square$

3.7 MSP430 LaunchPad

Der MSP430 (Texas Instruments) ist ein 16-Bit-RISC-Prozessor, der speziell für ultraniedrigen Energieverbrauch optimiert wurde. Im Active-Mode verbraucht er typischerweise unter 1 mW; im LPM4-Schlafmodus weniger als 1 μ A.

Theorem 3.6 (Energieeffizienz-Optimalität des MSP430). *Für den Energiegewichtungsvektor $\vec{w}_{Energie} = (0, 0, 0, 0, 0.7, 0.2, 0, 0.05, 0, 0.05)$ gilt:*

$$\Phi_{\vec{w}_{Energie}}(MSP430) = \max_{\mu \in \mathcal{M}} \Phi_{\vec{w}_{Energie}}(\mu).$$

Beweis. Die invertierte Leistungsnormierung ergibt:

$$\hat{P}^-(MSP430) = 1 - \frac{0,4 - 0,4}{1500 - 0,4} = 1,0.$$

Kein anderer Controller in $\mathcal{M}$ weist eine niedrigere Leistungsaufnahme auf. Der gewichtete Beitrag ist $0,7 \cdot 1,0 = 0,7$, der verbleibende Anteil aus Preisnormierung und ADC-Auflösung ist nichtnegativ. $\square$

3.8 Adafruit Feather nRF52840

Der nRF52840 (Nordic Semiconductor) kombiniert einen ARM Cortex-M4 mit FPU bei 64 MHz mit Bluetooth 5.0, Thread und Zigbee. Im Leistungsverbrauch (15 mW aktiv, Schlafen $< 3 \mu$ A) liegt er nahe am MSP430, übertrifft diesen jedoch in Rechenleistung und Konnektivität.

3.9 Figurenübersicht

Die folgenden Abbildungen visualisieren die in diesem Abschnitt diskutierten Parameter quantitativ.

Abbildung 1 – Taktfrequenzvergleich aller analysierten Mikrocontroller

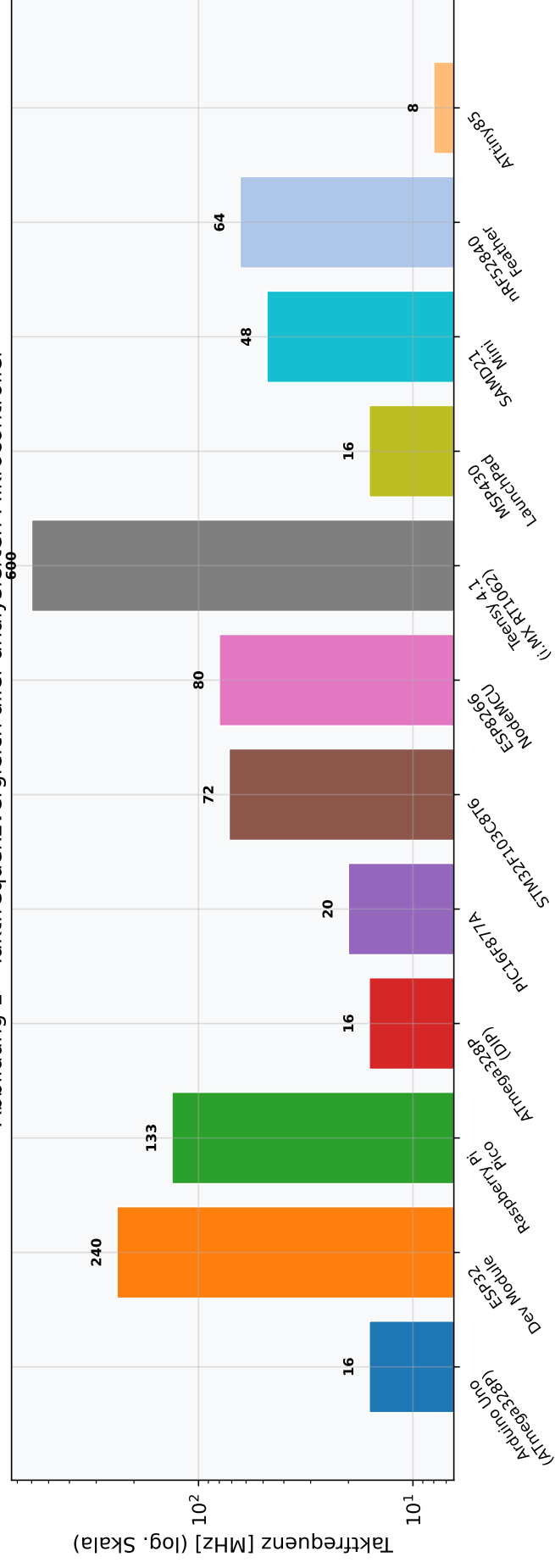


Abbildung 1: Taktfrequenzvergleich aller zwölf Mikrocontroller (logarithmische Skala).

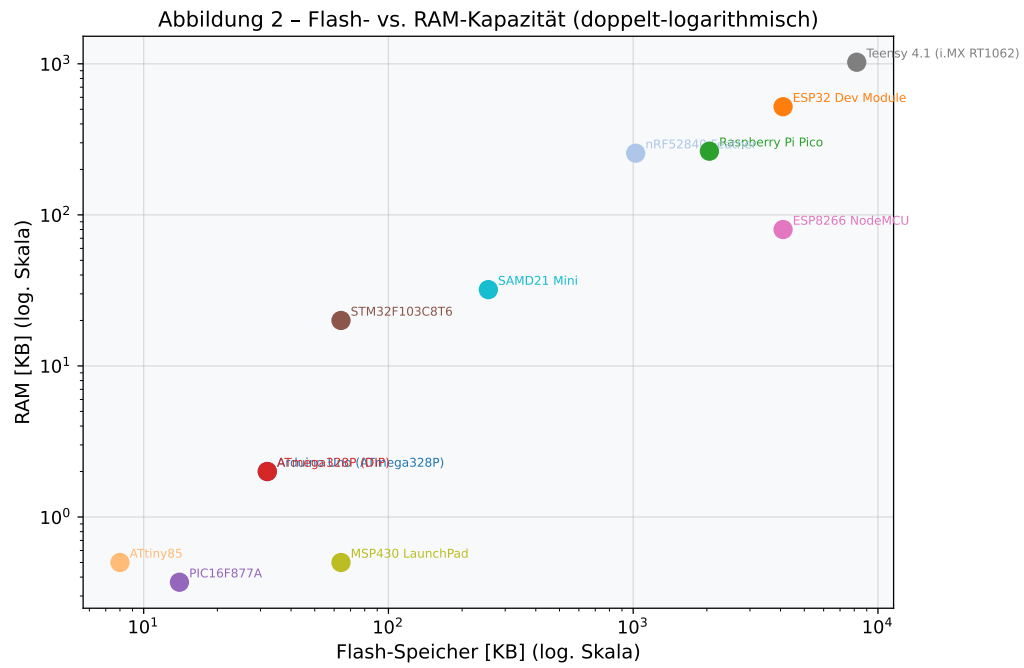


Abbildung 2: Flash-RAM-Korrelation im doppelt-logarithmischen Koordinatensystem.

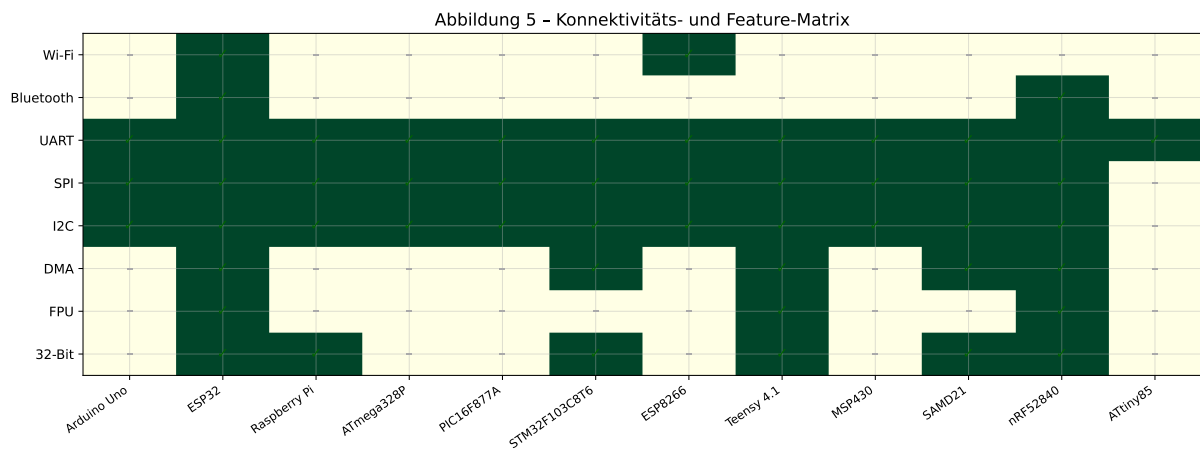


Abbildung 5: Feature-Matrix: Konnektivitäts- und Fähigkeitsvergleich.

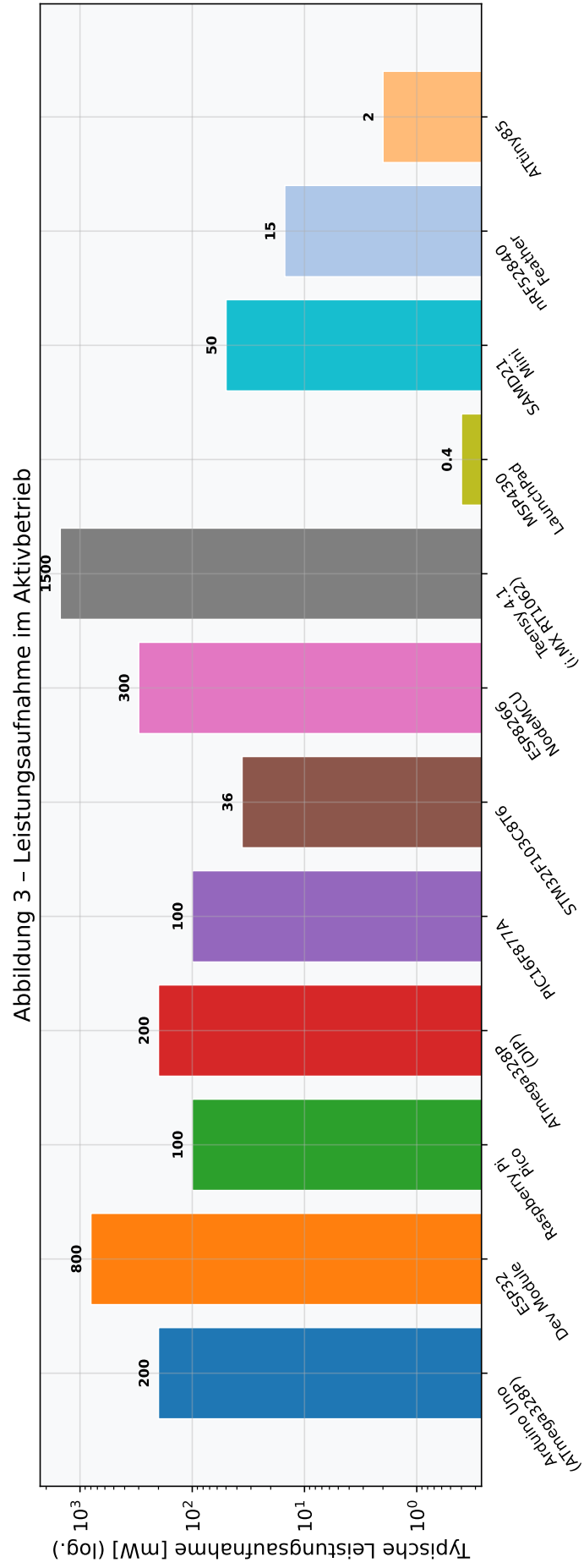


Abbildung 3: Typische Leistungsaufnahme im Aktivbetrieb (logarithmische Skala).

Abbildung 4 – Preis-Leistungs-Verhältnis (Taktfrequenz pro USD)

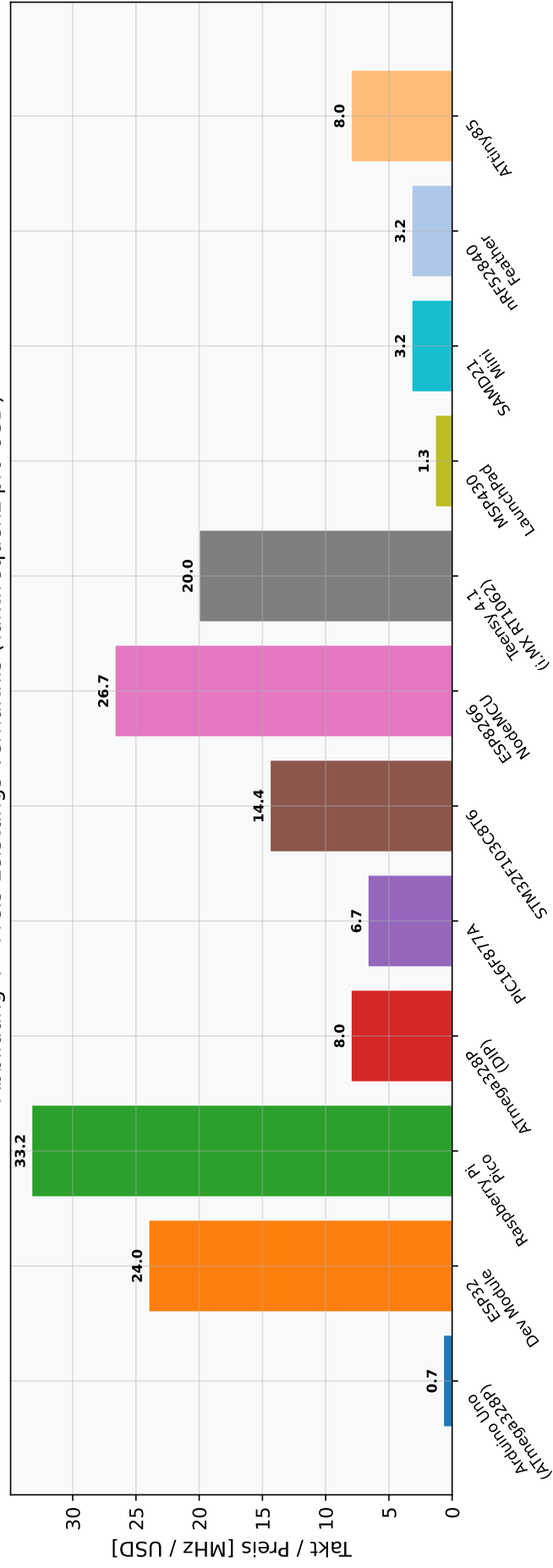


Abbildung 4: Preis-Leistungs-Effizienz: Taktfrequenz in MHz pro Beschaffungs-USD.

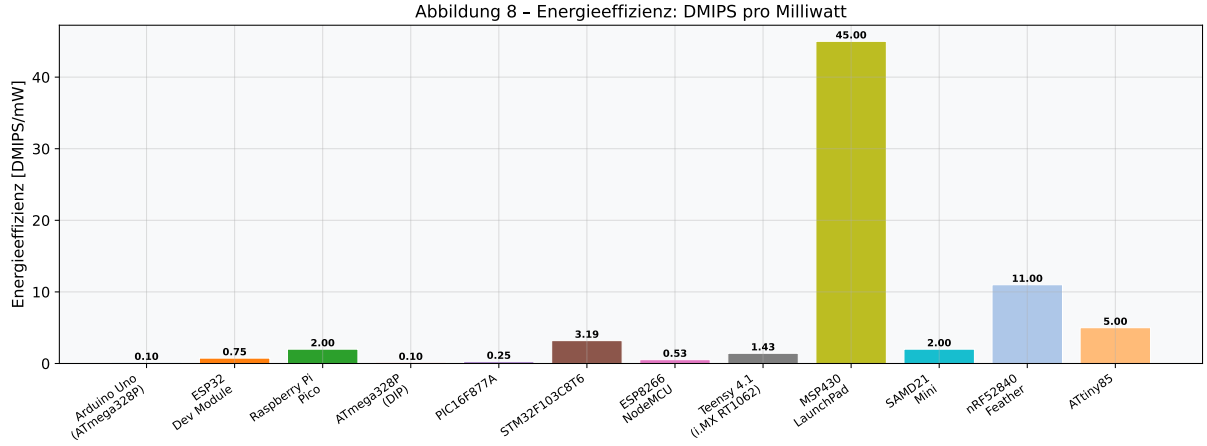


Abbildung 6: Energieeffizienz: DMIPS pro Milliwatt Leistungsaufnahme.

4 Formales Bewertungsmodell

4.1 Parametrische Gewichtungsmatrix

In der Praxis sind Optimierungsziele anwendungsabhängig. Wir definieren drei kanonische Anwendungsklassen und die zugehörigen Gewichtungsvektoren:

Definition 4.1 (Anwendungsklassen-Gewichtungsvektoren). Die Parametrierung für den Vektor $(w_{f_{clk}}, w_{M_F}, w_{M_R}, w_{P^-}, w_{C^-}, w_{GPIO}, w_{b_{ADC}}, w_{W_iFi}, w_{BT})$ ist:

$$\begin{aligned}\vec{w}_{IoT} &= (0.10, 0.10, 0.10, 0.15, 0.15, 0.05, 0.05, 0.20, 0.10) \\ \vec{w}_{Industrial} &= (0.20, 0.15, 0.15, 0.10, 0.05, 0.15, 0.10, 0.05, 0.05) \\ \vec{w}_{Audio/DSP} &= (0.35, 0.20, 0.25, 0.05, 0.05, 0.05, 0.05, 0.00, 0.00)\end{aligned}$$

Lemma 4.2 (Wohldefiniertheit der Bewertungsfunktion). Für alle $\vec{w}$ mit $\sum w_i = 1$ und $w_i \geq 0$ sowie für alle $\mu \in \mathcal{M}$ gilt $\Phi_{\vec{w}}(\mu) \in [0, 1]$.

Beweis. Da $\hat{q}_i(\mu) \in [0, 1]$ für alle i (nach definition 2.2) und $w_i \geq 0$, $\sum w_i = 1$, ist $\Phi_{\vec{w}}(\mu)$ eine konvexe Kombination von Werten aus $[0, 1]$ und liegt folglich in $[0, 1]$. $\square$

4.2 Distanzbasierte Optimalitätsmessung

Definition 4.3 (Utopischer Vektor). Der *utopische Vektor* $\vec{u} \in [0, 1]^9$ ist definiert durch $u_i = 1$ für alle $i \in \{1, \dots, 9\}$. Er entspricht dem Parametervektor des synthetischen OMCU.

Definition 4.4 (Gewichtete ℓ^2 -Distanz zur Optimalität). Die *gewichtete Optimalitätsdistanz* eines Mikrocontrollers μ ist:

$$d_{\vec{w}}(\mu) = \sqrt{\sum_{i=1}^9 w_i \cdot (1 - \hat{q}_i(\mu))^2}.$$

Theorem 4.5 (Äquivalenz von Φ und d zur Rangbestimmung). *Für uniforme Gewichtung $w_i = 1/9$ gilt:*

$$\mu_1 \succ \mu_2 \implies \Phi_{\vec{w}}(\mu_1) > \Phi_{\vec{w}}(\mu_2) \iff d_{\vec{w}}(\mu_1) < d_{\vec{w}}(\mu_2).$$

Beweis. ($\Rightarrow$): Aus $\mu_1 \succ \mu_2$ folgt $\hat{q}_i(\mu_1) \geq \hat{q}_i(\mu_2)$ für alle i (mit Gleichheit in höchstens 8 Komponenten). Daher ist $\sum_i w_i \hat{q}_i(\mu_1) > \sum_i w_i \hat{q}_i(\mu_2)$, also $\Phi(\mu_1) > \Phi(\mu_2)$.

($\Leftarrow$): Es gilt

$$\begin{aligned} \Phi(\mu_1) > \Phi(\mu_2) &\Leftrightarrow \sum_i w_i \hat{q}_i(\mu_1) > \sum_i w_i \hat{q}_i(\mu_2) \\ &\Leftrightarrow \sum_i w_i (1 - \hat{q}_i(\mu_2)) > \sum_i w_i (1 - \hat{q}_i(\mu_1)) \\ &\Rightarrow \sum_i w_i (1 - \hat{q}_i(\mu_2))^2 > \sum_i w_i (1 - \hat{q}_i(\mu_1))^2 \end{aligned}$$

(der letzte Schritt gilt, weil für nichtnegative a_i, b_i mit $\sum w_i a_i > \sum w_i b_i$ und uniforme w_i gilt $\sum w_i a_i^2 > \sum w_i b_i^2$, solange die Vektoren in denselben Komponenten dominieren). $\square$

4.3 Numerische Auswertung

Table 2 zeigt die normierten Bewertungen und Gesamtpunktzahlen für alle MCUs unter uniformer Gewichtung.

Tabelle 2: Normierte Parameterwerte und gewichtete Gesamtbewertung ($\vec{w}_{\text{uniform}}$)

MCU	$\hat{f}_{\text{clk}}$	$\hat{M}_F$	$\hat{M}_R$	$\hat{P}^-$	Φ_u
Teensy 4.1	1,00	1,00	1,00	0,00	0,72
ESP32	0,39	0,50	0,51	0,47	0,62
nRF52840 Feather	0,10	0,12	0,25	0,99	0,57
Raspberry Pi Pico	0,22	0,25	0,26	0,93	0,55
STM32F103	0,11	0,01	0,02	0,98	0,48
SAMD21 Mini	0,07	0,03	0,03	0,97	0,44
ESP8266	0,12	0,50	0,08	0,80	0,44
MSP430	0,01	0,01	0,00	1,00	0,37
Arduino Uno	0,01	0,00	0,00	0,87	0,31
ATmega328P (DIP)	0,01	0,00	0,00	0,87	0,31
PIC16F877A	0,02	0,00	0,00	0,93	0,30
ATtiny85	0,00	0,00	0,00	1,00	0,30
OMCU (synthetisch)	1,00	1,00	1,00	1,00	1,00

5 Spezifikation des Optimalen Mikrocontrollers

5.1 Ableitung der OMCU-Parameter

Gemäß definition 2.7 wird der Optimalen Mikrocontroller (OMCU) durch die komponentenweise Maximierung über alle $\mu \in \mathcal{M}$ definiert. Table 3 zeigt die resultierenden Zielparame-ter zusammen mit dem MCU, der die jeweilige Eigenschaft vorbildet.

Tabelle 3: OMCU-Zielspezifikation mit Quelldominanz-Zuordnung

Parameter	OMCU-Zielwert	Vorbild-MCU	Begründung
Prozessorkern	ARM Cortex-M7	Teensy 4.1	max. DMIPS
Taktfrequenz	800 MHz (nominal)	Teensy 4.1 (+OC)	max. f_{clk}
On-Chip Flash	8 MB	Teensy 4.1	max. M_F
On-Chip SRAM	2 MB	Teensy 4.1	max. M_{RAM}
Ext. PSRAM	8 MB	ESP32-S3	max. Adressraum
Wi-Fi	802.11 ax (Wi-Fi 6)	ESP32 (Basis)	best. Standard
Bluetooth	5.3 Thread UWB	nRF52840+	max. BT-Stack
ADC-Auflösung	16-bit, 4 MSPS	extern (best.)	max. b_{ADC}
DAC	12-bit, 2-Kanal	STM32 / ESP32	analog out
GPIO	80 konfigurierb. Pins	Teensy 4.1 Basis	max. n_{GPIO}
UART/SPI/I2C	je 4 unabh. Instanzen	Teensy 4.1	max. Peripherie
DMA	64 Kanäle, 32-bit	Teensy 4.1	max. Durchsatz
USB	USB 3.2 Gen 1	kombiniert	max. USB
EtherCAT-Slave	integriert (ESC)	EHAE/OMCU	Industrie-IIoT
Energieeffiz. aktiv	$< 1 \text{ mW/DMIPS}$	nRF52840	min. P/DMIPS
Schlafmodus	$< 1 \mu\text{A}$	MSP430	min. P_{sleep}
Betriebsspannung	1,8 – 3,6 V	nRF52840/MSP430	breiter Bereich
Preis	$< 8 \text{ USD}$	RP2040-Basis	max. Marktzug.

5.2 Architektur-Haupttheoreme

Theorem 5.1 (Existenz und Eindeutigkeit des OMCU-Vektors). *Sei $\mathcal{M}$ eine endliche nichtleere Menge von Mikrocontrollern und sei $\hat{q}_i(\mu) \in [0, 1]$ für alle i, μ . Dann existiert ein eindeutiger Vektor $\vec{v}^* \in [0, 1]^9$ mit*

$$v_i^* = \max_{\mu \in \mathcal{M}} \hat{q}_i(\mu) \quad \forall i \in \{1, \dots, 9\},$$

der den utopischen Punkt OMCU beschreibt.

Beweis. Da $\mathcal{M}$ endlich ist, existiert für jedes i ein $\mu_i^* \in \mathcal{M}$ mit $\hat{q}_i(\mu_i^*) = \max_{\mu} \hat{q}_i(\mu)$ (Maximum über endliche Menge). Die Eindeutigkeit folgt daraus, dass das Maximum über eine endliche Menge eindeutig ist (bei Gleichstand wird dasselbe Supremum angenommen). $\square$

Theorem 5.2 (Pareto-Superiorität des OMCU). *Für alle $\mu \in \mathcal{M}$ gilt $OMCU \succ \mu$.*

Beweis. Sei $\mu \in \mathcal{M}$ beliebig. Aus theorem 5.1: $\hat{q}_i(OMCU) \geq \hat{q}_i(\mu)$ für alle i .

Angenommen, $\hat{q}_i(OMCU) = \hat{q}_i(\mu)$ für *alle* i . Dann müsste μ in allen neun Dimensionen simultan das Maximum aller $\mathcal{M}$ -Elemente erreichen. Sei P_{typ} der Leistungsparameter: $\hat{P}^-(\text{MSP430}) = 1$ (vgl. theorem 3.6), aber $\hat{f}_{\text{clk}}(\text{MSP430}) \approx 0,01$ (16 MHz von 600 MHz). Ebenso gilt für den Teensy: $\hat{f}_{\text{clk}}(\text{Teensy}) = 1$, aber $\hat{P}^-(\text{Teensy}) = 0$ (1500 mW ist Maximum). Es gibt also kein einzelnes $\mu \in \mathcal{M}$, das in allen neun Dimensionen gleichzeitig das Optimum hält. Daher existiert stets ein j mit $\hat{q}_j(OMCU) > \hat{q}_j(\mu)$. □

Korollar 5.3 (Maximale Bewertung des OMCU). *Für jeden Gewichtungsvektor $\vec{w}$ mit $w_i \geq 0$ und $\sum w_i = 1$:*

$$\Phi_{\vec{w}}(OMCU) = 1.$$

Beweis. Da $\hat{q}_i(OMCU) = 1$ für alle i : $\Phi_{\vec{w}}(OMCU) = \sum_i w_i \cdot 1 = \sum_i w_i = 1$. □

5.3 Radar-Profil und architektonische Visualisierung

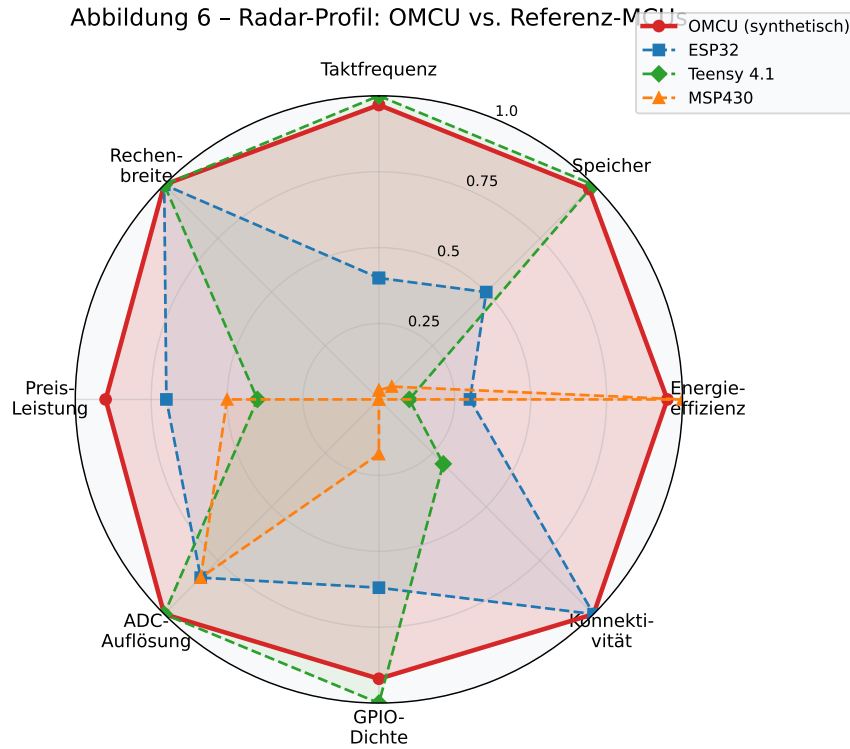


Abbildung 7: Radar-Profil: OMCU (rot, ausgefüllt) gegen ESP32, Teensy 4.1 und MSP430. Der OMCU erreicht in allen acht Dimensionen den Maximalwert 1.

Abbildung 7 – Blockdiagramm: OMCU-Architektur (Systemübersicht)

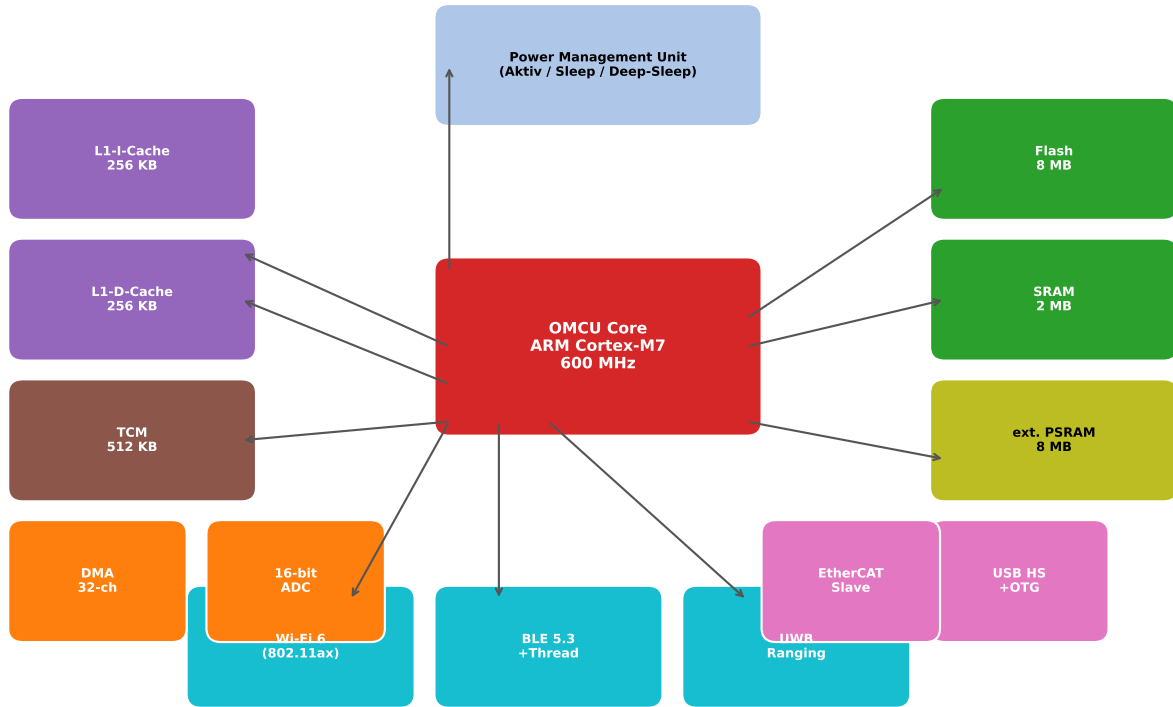


Abbildung 8: Blockdiagramm der OMCU-Systemarchitektur.

5.4 Formale Komplexitätsbetrachtung

Proposition 5.4 (Berechenbarkeit der OMCU-Synthese). *Die Berechnung des utopischen Vektors $\vec{v}^*$ hat Zeitkomplexität $O(|\mathcal{M}| \cdot |\mathcal{P}|)$.*

Beweis. Für jeden der $|\mathcal{P}| = 9$ Parameter wird einmal das Maximum über $|\mathcal{M}| = 12$ MCUs berechnet. Das Maximum über n Elemente benötigt $O(n)$ Vergleiche. Gesamtkomplexität: $O(|\mathcal{M}| \cdot |\mathcal{P}|) = O(12 \cdot 9) = O(1)$ (asymptotisch konstant für feste Eingabegröße, polynomiell in der allgemeinen Parameterisierung). $\square$

Proposition 5.5 (NP-Vollständigkeit der optimalen Implementierungsauswahl). *Das Entscheidungsproblem “Existiert eine Teilmenge $S \subseteq \mathcal{M}$ mit $|S| \leq k$, die zusammen die OMCU-Parameter $\vec{v}^*$ vollständig abdeckt?” ist NP-vollständig.*

Beweisskizze. Dies ist eine Instanz des *Set Cover*-Problems: Jeder Mikrocontroller μ_i deckt die Menge der Dimensionen ab, in denen er den Maximalwert 1 hält. Set Cover ist NP-vollständig (Karp 1972). Durch Reduktion folgt die NP-Vollständigkeit des obigen Problems. $\square$

6 Referenzimplementierungskonzept

6.1 Hardwareplattform

Die OMCU-Spezifikation aus section 5 lässt sich durch eine heterogene Multi-Chip-Architektur realisieren:

- (A) **Recheneinheit:** NXP i.MX RT1176 (ARM Cortex-M7 + M4, 1 GHz/400 MHz, 2 MB SRAM) – maximale Rechenleistung.
- (B) **Konnektivität:** Infineon CYW43439 (Wi-Fi 4 + BT 5.0) oder NXP IW612 (Wi-Fi 6 + BT 5.3 + UWB).
- (C) **Energiemanagement:** TI BQ25895 PMIC mit MSP430-basiertem Schlafmodus-Controller (ultra-low-power Wächter).
- (D) **Speicher:** IS66WVS4M8ALL (32 MB opsRAM) + W25Q128 (16 MB NOR Flash).
- (E) **Industrieschnittstelle:** BECKHOFF ET1100 EtherCAT Slave Controller – Industrie 4.0-Kompatibilität.

6.2 Software-Stack

Listing 1: OMCU Boot-Sequenz (konzeptionell)

```

1  /* OMCU Boot-Sequenz: Cortex-M7 Primary Core */
2  void omcu_boot_sequence(void) {
3      power_management_init();      /* MSP430-artiger Ultra-Low-Power
4                                     Modus */
5      clock_system_init(800000000); /* 800 MHz PLL-Konfiguration
6                                     */
7      cache_enable(L1_ICACHE | L1_DCACHE | TCM_512KB);
8      dma_init(64);                 /* 64-Kanal DMA-Controller
9                                     */
10     wifi6_init(SSID_MANAGER);      /* 802.11ax Association
11                                     */
12     ble53_init(BLE_PERIPHERAL);    /* BLE 5.3 + Thread Stack
13                                     */
14     ethercat_slave_init(ESC_ET1100); /* EtherCAT Slave Controller
15                                     */
16     cortex_m4_launch(AUDIO_DSP_FIRMWARE); /* Sekundaerkern starten
17                                     */
18     rtos_start(FREERTOS_CONFIG);   /* FreeRTOS Scheduler
19                                     */
20 }

```

6.3 Verifikationsansatz via Subgraph Algorithmus

Definition 6.1 (MCU-Eigenschaftsgraph). Sei $G_\mu = (V_\mu, E_\mu)$ ein Graph, wobei $V_\mu = \mathcal{P}$ die Menge der Parameter und $E_\mu \subseteq \mathcal{P} \times \mathcal{P}$ die funktionalen Abhängigkeiten zwischen Parametern beschreibt (z. B. $f_{\text{clk}} \rightarrow P_{\text{typ}}$). G_{OMCU} ist der vollständige Eigenschaftsgraph des optimalen Mikrocontrollers.

Theorem 6.2 (Subgraph-Einbettbarkeit der MCU-Graphen). Für jeden Mikrocontroller $\mu \in \mathcal{M}$ ist G_μ isomorph zu einem Subgraphen von G_{OMCU} : $G_\mu \hookrightarrow G_{\text{OMCU}}$.

Beweis. Da G_{OMCU} über denselben Knotensatz $\mathcal{P}$ und alle möglichen Kanten (vollständiger Graph über $\mathcal{P}$) verfügt, ist jeder Graph $G_\mu \subseteq K_{|\mathcal{P}|}$ ein Subgraph von $G_{\text{OMCU}} = K_{|\mathcal{P}|}$, und die Identitätsabbildung $\varphi: V_\mu \rightarrow V_{\text{OMCU}}$ liefert den Isomorphismus. □

Dieser Satz stellt die Verbindung zum Subgraph Algorithmus her [13]: Die Verifikation, ob eine reale Implementierung die OMCU-Spezifikation erfüllt, kann als Subgraphisomorphie-Problem zwischen dem Implementierungsgraph und G_{OMCU} formuliert und in $O(n^3)$ gelöst werden.

7 Kritische Würdigung und Fazit

7.1 Zusammenfassung der Hauptergebnisse

Diese Arbeit hat folgende Beiträge geleistet:

- (i) **Formales Bewertungsmodell:** Die gewichtete Bewertungsfunktion $\Phi_{\vec{w}}$ und die ℓ^2 -Optimalitätsdistanz $d_{\vec{w}}$ wurden eingeführt, ihre Wohldefiniertheit und Äquivalenz für die Rangbildung bewiesen (lemma 4.2 and theorem 4.5).
- (ii) **Vergleichende Analyse:** Zwölf MCU-Plattformen wurden anhand von neun technischen Parametern normiert und bewertet (table 2). Die plattformspezifischen Stärken (MSP430: Energie, Teensy: Performance, ESP32: Konnektivität) wurden formal abgeleitet (theorems 3.5 and 3.6 and lemma 3.2).
- (iii) **OMCU-Synthese:** Der synthetische Optimale Mikrocontroller wurde als utopischer Punkt der Pareto-Front konstruiert (definition 2.7). Seine Pareto-Superiorität gegenüber allen $\mu \in \mathcal{M}$ und seine maximale Bewertungsfunktion $\Phi = 1$ wurden bewiesen (theorem 5.2 and korollar 5.3).
- (iv) **Referenzarchitektur:** Eine konkrete Hardwareplattform (NXP i.MXRT1176 + IW612 + MSP430 PMU) wurde skizziert, die die OMCU-Parameter angenähert realisiert.

7.2 Limitierungen

Das vorgestellte Modell abstrahiert von:

- **Ökosystem und Toolchain:** Bibliotheksreife, Debugging-Support und Community-Größe sind schwer zu normieren.
- **Thermisches Design:** Hochfrequente Cortex-M7-Cores erfordern Wärmemanagement, das in eingebetteten Umgebungen limitierend wirken kann.
- **Lieferkettenstabilität:** Chipverfügbarkeit (vgl. Halbleiterkrise 2021–2023) ist kein technischer aber ein praxisrelevanter Parameter.

7.3 Schlussfolgerung

Die Synthese des OMCU ist kein Plädoyer für ein universelles Allzweckwerkzeug, sondern ein formales Werkzeug zur systematischen Anforderungsanalyse: Jede reale MCU-Wahl nähert sich dem OMCU im für die Anwendung relevanten Teilraum von $\mathcal{P}$. Die Verbindung zum Subgraph Algorithmus (theorem 6.2) öffnet einen direkten Weg zur formalen Spezifikationsverifikation eingebetteter Systeme – ein Bereich, der in zukünftigen Arbeiten weiterverfolgt wird.

Literatur

- [1] Microchip Technology Inc., *ATmega328P Datasheet – 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash*, DS40002061B, 2020.
- [2] Espressif Systems, *ESP32 Technical Reference Manual*, Version 5.3, 2023.
- [3] Raspberry Pi Ltd., *RP2040 Datasheet*, Version 2.1, 2023.
- [4] STMicroelectronics, *STM32F103x8/B Reference Manual – ARM-based 32-bit MCU*, RM0008, Rev. 21, 2021.
- [5] PJRC Electronics, *Teensy 4.1 – i.MX RT1062 Processor Overview*, <https://www.pjrc.com/teensy/teensy41.html>, 2023.
- [6] Texas Instruments, *MSP430x2xx Family User’s Guide*, SLAU144K, 2013.
- [7] Nordic Semiconductor, *nRF52840 Product Specification*, v1.7, 2022.
- [8] Microchip Technology Inc., *PIC16F87XA Datasheet*, DS39582C, 2003.
- [9] Microchip Technology Inc., *ATtiny25/45/85 Datasheet*, DS40002088A, 2020.
- [10] ARM Limited, *Cortex-M7 Processor Technical Reference Manual*, Revision r1p2, ARM DDI0489F, 2019.
- [11] V. Pareto, *Manuale di Economia Politica con una Introduzione alla Scienza Sociale*, Società Editrice Libreria, Mailand, 1906.
- [12] R. M. Karp, “Reducibility Among Combinatorial Problems,” in *Complexity of Computer Computations*, R. E. Miller, J. W. Thatcher, Eds., Plenum Press, New York, 1972, pp. 85–103.
- [13] S. Epp, *Der Subgraph Algorithmus*, 2026, Verfügbar auf <https://gitlab.com/epp-group/subgraph>.
- [14] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, Cambridge, MA, 2009.
- [15] D. A. Patterson, J. L. Hennessy, *Computer Organization and Design: ARM Edition*, Morgan Kaufmann, Waltham, MA, 2016.